

# TEORI BAHASA DAN AUTOMATA

## Modul Pembelajaran: Epsilon NFA ( $\epsilon$ -NFA)

Penjelasan mendalam mengenai transisi gaib (epsilon), cara menghitung epsilon-closure, dan trik merancang mesin automata yang fleksibel dengan banyak studi kasus.

### Contents

|          |                                                                          |          |
|----------|--------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Mengenal Epsilon NFA (<math>\epsilon</math>-NFA)</b>                  | <b>2</b> |
| 1.1      | Apa Itu Transisi Epsilon?                                                | 2        |
| 1.2      | Kenapa Kita Butuh $\epsilon$ -NFA?                                       | 2        |
| <b>2</b> | <b>Definisi Formal dan <math>\epsilon</math>-Closure</b>                 | <b>3</b> |
| 2.1      | Tupel Matematis $\epsilon$ -NFA                                          | 3        |
| 2.2      | Kunci Utama: $\epsilon$ -Closure                                         | 3        |
| <b>3</b> | <b>Kumpulan Studi Kasus Perancangan <math>\epsilon</math>-NFA</b>        | <b>4</b> |
| 3.1      | Kasus 1: Pilihan Paralel (Kondisi ATAU)                                  | 4        |
| 3.2      | Kasus 2: Elemen Opsional (Boleh Ada, Boleh Nggak)                        | 4        |
| 3.3      | Kasus 3: Perulangan Penuh (Kleene Star)                                  | 5        |
| 3.4      | Kasus 4: Penggabungan Dua Mesin (Concatenation)                          | 5        |
| <b>4</b> | <b>Cara Mengeksekusi / Tracing String pada <math>\epsilon</math>-NFA</b> | <b>6</b> |

# 1 Mengenal Epsilon NFA ( $\epsilon$ -NFA)

## 1.1 Apa Itu Transisi Epsilon?

Kalau di materi NFA biasa kita udah tahu kalau mesin bisa bercabang atau punya jalan buntu, sekarang kita kenalan sama kekuatan utamanya: **Transisi Epsilon** ( $\epsilon$ ). Di beberapa literatur, simbol ini kadang ditulis pakai lambda ( $\lambda$ ).

Bayangin transisi epsilon ini kayak **jalan pintas atau pintu teleportasi**. Biasanya, buat pindah dari satu *state* ke *state* lain, mesin butuh "bayaran" berupa ngebaca satu karakter dari pita input. Nah, dengan panah  $\epsilon$ , mesin diizinkan pindah ke *state* tujuan **secara gratis**, tanpa perlu ngebaca, menggeser, atau mengonsumsi huruf apa pun dari pita!

### Analogi Epsilon ( $\epsilon$ )

Anggap kamu lagi main *board game*. Biasanya buat maju 1 langkah, kamu harus lempar dadu dulu. Transisi  $\epsilon$  ini ibarat kartu "Bebas Melangkah". Begitu kamu dapet jalan yang ada  $\epsilon$ -nya, kamu bisa langsung loncat ke petak berikutnya tanpa perlu nunggu giliran lempar dadu.

## 1.2 Kenapa Kita Butuh $\epsilon$ -NFA?

Mungkin kamu mikir, "NFA biasa aja udah cukup kok, ngapain nambahin teleportasi segala?". Alasan terbesarnya adalah **Modularitas**.

Bikin mesin DFA atau NFA biasa untuk mendeteksi bahasa yang super panjang dan ribet itu pusing banget. Dengan  $\epsilon$ -NFA, kita bisa bikin mesin-mesin kecil yang tugasnya sederhana, lalu kita jahit (sambungkan) mesin-mesin kecil itu pakai jalan tol  $\epsilon$ . Konsep ini sangat berguna pas kita belajar tentang *Regular Expression* (Regex), karena setiap pola Regex pada dasarnya dibangun dari potongan-potongan  $\epsilon$ -NFA yang digabungkan.

## 2 Definisi Formal dan $\epsilon$ -Closure

### 2.1 Tupel Matematis $\epsilon$ -NFA

Secara matematika,  $\epsilon$ -NFA tetep pakai 5-tupel yang sama kayak NFA biasa:  $M = (Q, \Sigma, \delta, q_0, F)$ . Bedanya cuma ada di satu tempat, yaitu di **\*\*Fungsi Transisinya ( $\delta$ )\*\***.

Pemetaan fungsi transisinya ditulis:

$$\delta : Q \times (\Sigma \cup \{\epsilon\}) \rightarrow \mathcal{P}(Q)$$

Artinya: Sebuah *state* kalau dapet input dari alfabet ( $\Sigma$ ) **ATAU** dapet input kosong ( $\epsilon$ ), dia bakal menghasilkan himpunan *state* tujuan ( $\mathcal{P}(Q)$ ).

### 2.2 Kunci Utama: $\epsilon$ -Closure

Ini adalah konsep paling penting kalau kamu mau ngerjain soal atau merancang  $\epsilon$ -NFA.  **$\epsilon$ -Closure** (dibaca: epsilon-klosur) dari sebuah *state* adalah kumpulan semua *state* yang bisa didatengin dari *state* tersebut **hanya dengan menggunakan transisi  $\epsilon$** .

Aturan main  $\epsilon$ -Closure:

1. Sebuah *state* **selalu** bisa mendatangi dirinya sendiri dengan biaya nol. Jadi, nama *state* itu pasti masuk ke dalam himpunan  $\epsilon$ -Closure-nya sendiri.
2. Kalau dari *state* A ada jalan  $\epsilon$  ke B, dan dari B ada jalan  $\epsilon$  ke C, maka  $\epsilon$ -Closure dari A meliputi A, B, dan C. Sifatnya menjalar kayak air tumpah!

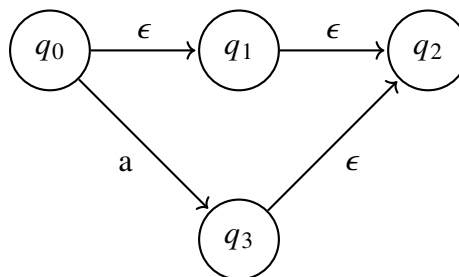


Figure 1: Contoh Sederhana untuk Menghitung  $\epsilon$ -Closure

Mari kita hitung  $\epsilon$ -Closure dari diagram di atas:

- $\epsilon\text{-Cl}(q_0) = \{q_0, q_1, q_2\}$  (Bisa ke dirinya sendiri, loncat ke  $q_1$ , dan karena  $q_1$  nyambung ke  $q_2$ , dia ikut tembus ke  $q_2$ ).
- $\epsilon\text{-Cl}(q_1) = \{q_1, q_2\}$
- $\epsilon\text{-Cl}(q_2) = \{q_2\}$  (Nggak ada panah  $\epsilon$  yang keluar dari dia).
- $\epsilon\text{-Cl}(q_3) = \{q_3, q_2\}$

### 3 Kumpulan Studi Kasus Perancangan $\epsilon$ -NFA

Di bagian ini kita bakal lihat betapa fleksibelnya  $\epsilon$ -NFA buat nyelesaiin masalah-masalah yang rumit kalau dikerjain pakai DFA.

#### 3.1 Kasus 1: Pilihan Paralel (Kondisi ATAU)

Kita mau bikin mesin buat mendeteksi string yang memuat "11" berurutan **ATAU** "00" berurutan. Ini kasus klasik di mana  $\epsilon$ -transition sangat bersinar.

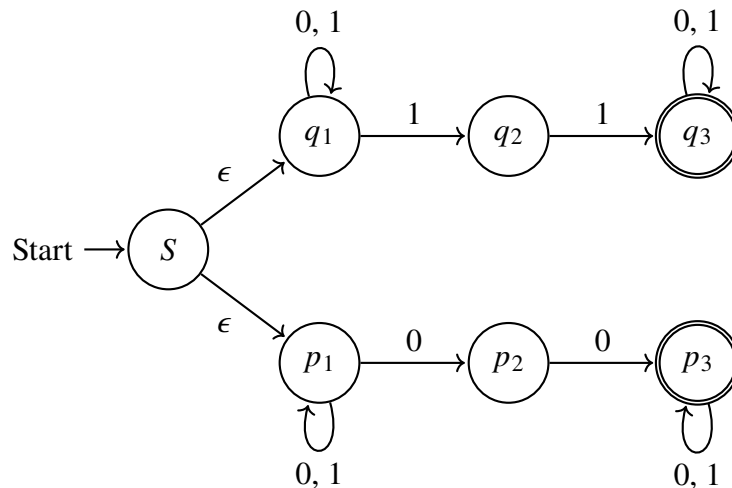


Figure 2: Cabang Paralel menggunakan  $\epsilon$ -NFA

Begitu mesin nyala, kloningannya langsung pecah dua secara otomatis. Satu tim di atas fokus mantau pola "11", satu tim di bawah fokus mantau pola "00". Kalau salah satu tim berhasil nyampe *Final State*, string langsung diterima. Nggak perlu repot bikin panah silang antar *state*.

#### 3.2 Kasus 2: Elemen Opsional (Boleh Ada, Boleh Nggak)

Misalkan kita mau mendeteksi penulisan angka desimal yang boleh pakai tanda plus (+), minus (-), atau **nggak pakai tanda sama sekali**. Alfabetnya  $\Sigma = \{+, -, d\}$  (anggap aja 'd' itu singkatan dari digit angka apa pun).

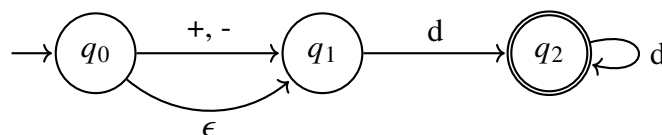


Figure 3: Transisi  $\epsilon$  untuk *Bypassing* (Melewati) State

Di sini, panah  $\epsilon$  berfungsi sebagai jalan *bypass*.

- Kalau inputnya "+d" atau "-d", mesin bakal milih jalur normal baca simbol tanda, lalu masuk ke  $q_1$ , lalu baca 'd' dan diterima di  $q_2$ .
- Kalau inputnya cuma "d" (tanpa tanda), mesin tinggal pilih jalan tol  $\epsilon$  buat langsung lompat ke  $q_1$  tanpa baca apa-apa, lalu baca 'd' dan tetep diterima dengan sukses!

### 3.3 Kasus 3: Perulangan Penuh (Kleene Star)

Kasus ini paling sering nongol di Regex. Kita mau terima bahasa  $a^*b^*c^*$ . Artinya: huruf 'a' dulu (boleh nggak ada), lanjut huruf 'b' (boleh nggak ada), ditutup huruf 'c' (boleh nggak ada).

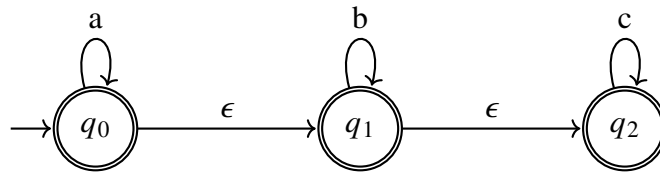


Figure 4: Penggunaan  $\epsilon$ -NFA untuk Perulangan Bersambung

Karena semua *state* adalah *Final State*, dan disatukan pakai panah  $\epsilon$ , kita bisa ngerasain fleksibilitas yang luar biasa:

- Kalau kita masukin string kosong (nggak ngetik apa-apa), posisinya tetep di  $q_0$  (Final)  $\rightarrow$  Diterima.
- Kalau masukin "ac", mesin baca 'a' di  $q_0$ , terus seketika pindah pakai  $\epsilon$  lewatin  $q_1$  dan langsung ke  $q_2$  buat baca 'c'  $\rightarrow$  Diterima.
- Kalau masukin "ca", mesin bakal meluncur ke  $q_2$  buat baca 'c', tapi pas mau baca 'a' dia gagal karena nggak ada panah mundur  $\rightarrow$  Ditolak.

### 3.4 Kasus 4: Penggabungan Dua Mesin (Concatenation)

Anggaplah kita punya dua mesin terpisah. Mesin pertama nyari pola yang huruf terakhirnya "a". Mesin kedua nyari pola yang huruf pertamanya "b". Kita pengen gabungin mesin ini biar dia terima kata yang ada transisi "ab" di tengahnya.

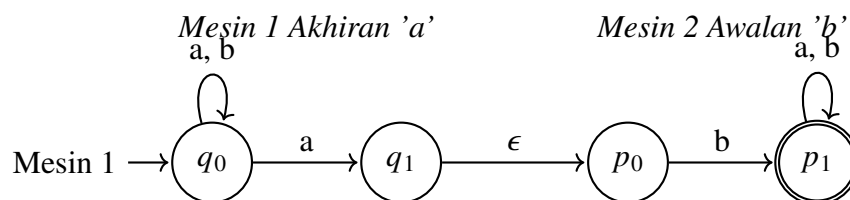


Figure 5: Penggabungan Mesin menggunakan Jembatan  $\epsilon$

Transisi  $\epsilon$  di sini bertindak sebagai **\*\*jembatan\*\***. Tanpa  $\epsilon$ , kita harus narik ulang panah-panah dengan rumus yang ribet. Pakai  $\epsilon$ , kita tinggal ambil *Final State* dari Mesin 1 (hapus status finalnya), tarik garis  $\epsilon$ , tempel ke *Initial State* Mesin 2. Beres!

## 4 Cara Mengeksekusi / Tracing String pada $\epsilon$ -NFA

Di bagian ini, kita bahas gimana sih cara ngebaca atau memproses string kalau ada  $\epsilon$ -nya? Logika jalannya mesin adalah: **\*\*Tutup dulu pakai Epsilon, baru Baca, terus Tutup lagi\*\***.

Secara urutan komputasi:

1. Hitung  $\epsilon$ -Closure dari *state* saat ini. Ini nentuin kloningan awal kita ada di mana aja.
2. Dari semua kloningan itu, baca 1 karakter input dan gerak sesuai panahnya.
3. Setelah pindah, hitung LAGI  $\epsilon$ -Closure dari tempat-tempat baru tersebut.

### Contoh Simulasi Tracing:

Kita pakai mesin di Kasus 3 (Pola  $a^*b^*c^*$ ) dan kita masukan string **"bc"**.

- **Start:** Posisi awal di  $q_0$ . Jangan lupa langsung hitung  $\epsilon$ -Closure nya.  
 $\epsilon\text{-Cl}(q_0) = \{q_0, q_1, q_2\}$ .  
 Artinya, sebelum baca karakter apa pun, mesin kita udah ada di 3 tempat sekaligus!
- **Baca Karakter Pertama 'b':**  
 Kita tanya satu-satu: -  $q_0$  baca 'b'? Gagal (jalan buntu).  
 -  $q_1$  baca 'b'? Tembus ke  $q_1$ .  
 -  $q_2$  baca 'b'? Gagal (jalan buntu).  
 Hasil bacanya cuma ada di  $q_1$ . Langsung kita hitung penutupnya:  $\epsilon\text{-Cl}(q_1) = \{q_1, q_2\}$ .  
 Posisi mesin sekarang setelah baca 'b' ada di  $\{q_1, q_2\}$ .
- **Baca Karakter Kedua 'c':**  
 Kita tanya lagi dari posisi sekarang: -  $q_1$  baca 'c'? Gagal (jalan buntu).  
 -  $q_2$  baca 'c'? Tembus ke  $q_2$ .  
 Hasil bacanya cuma ada di  $q_2$ . Tutup lagi:  $\epsilon\text{-Cl}(q_2) = \{q_2\}$ .

String habis! Posisi akhir mesin ada di himpunan  $\{q_2\}$ . Karena  $q_2$  adalah *Final State*, maka string "bc" dinyatakan **Diterima**.